19.12.23

OPENCAMPUS.sh

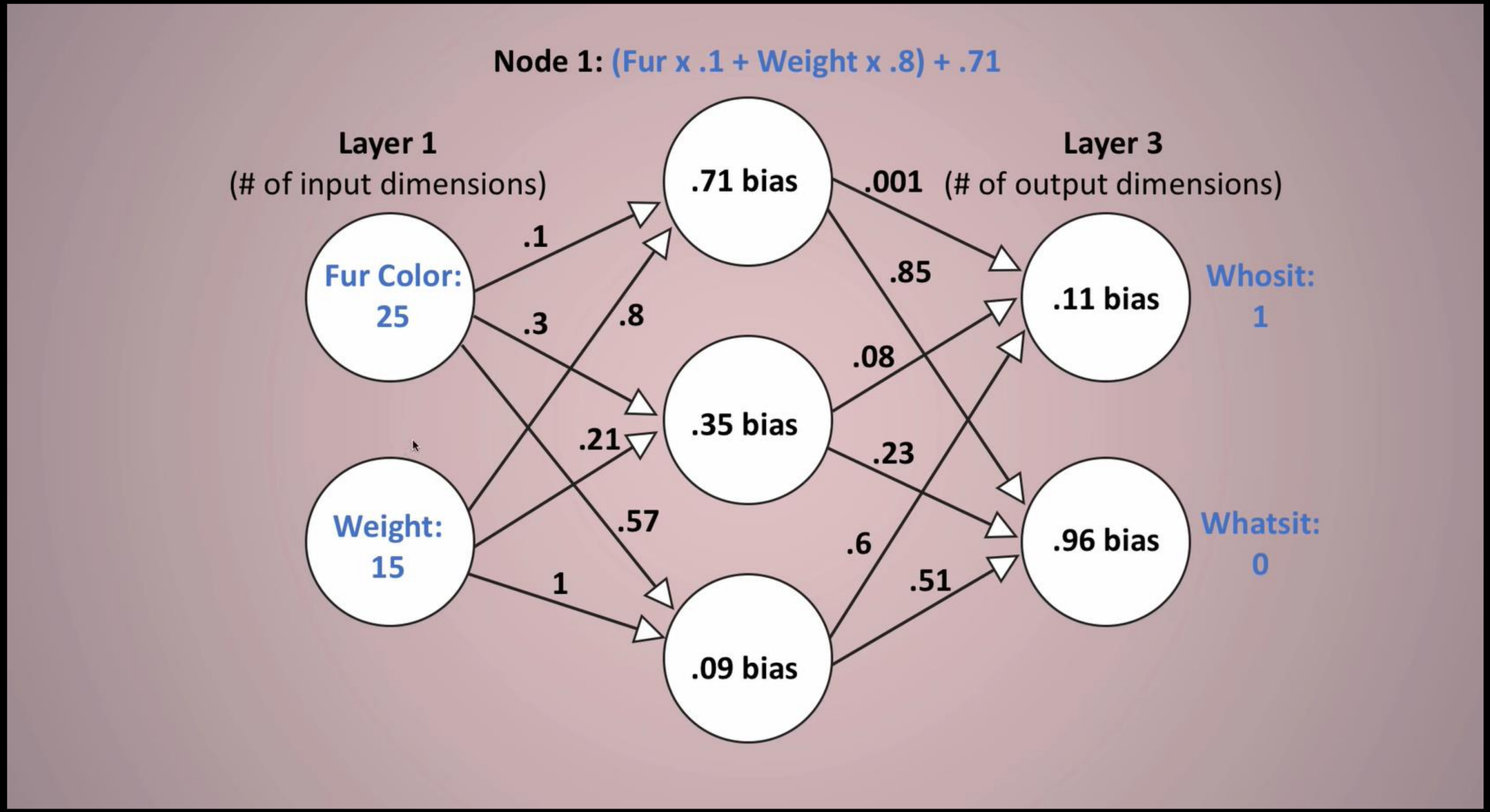
Einführung in Data Science und maschinelles Lernen

NEURONALE NETZE



- Quiz
- Neuronale Netze (NN)
- Hyperparameter in NN
- Frameworks zur Implementierung von NN
- Implementierung eines NN mit TensorFlow und Python

QUIZ



LearnCode.academy. (2018, June 11). *Neural Networks Explained—Machine Learning Tutorial for Beginners*. https://www.youtube.com/watch?v=GvQwE2OhL8I

Learning Rate:

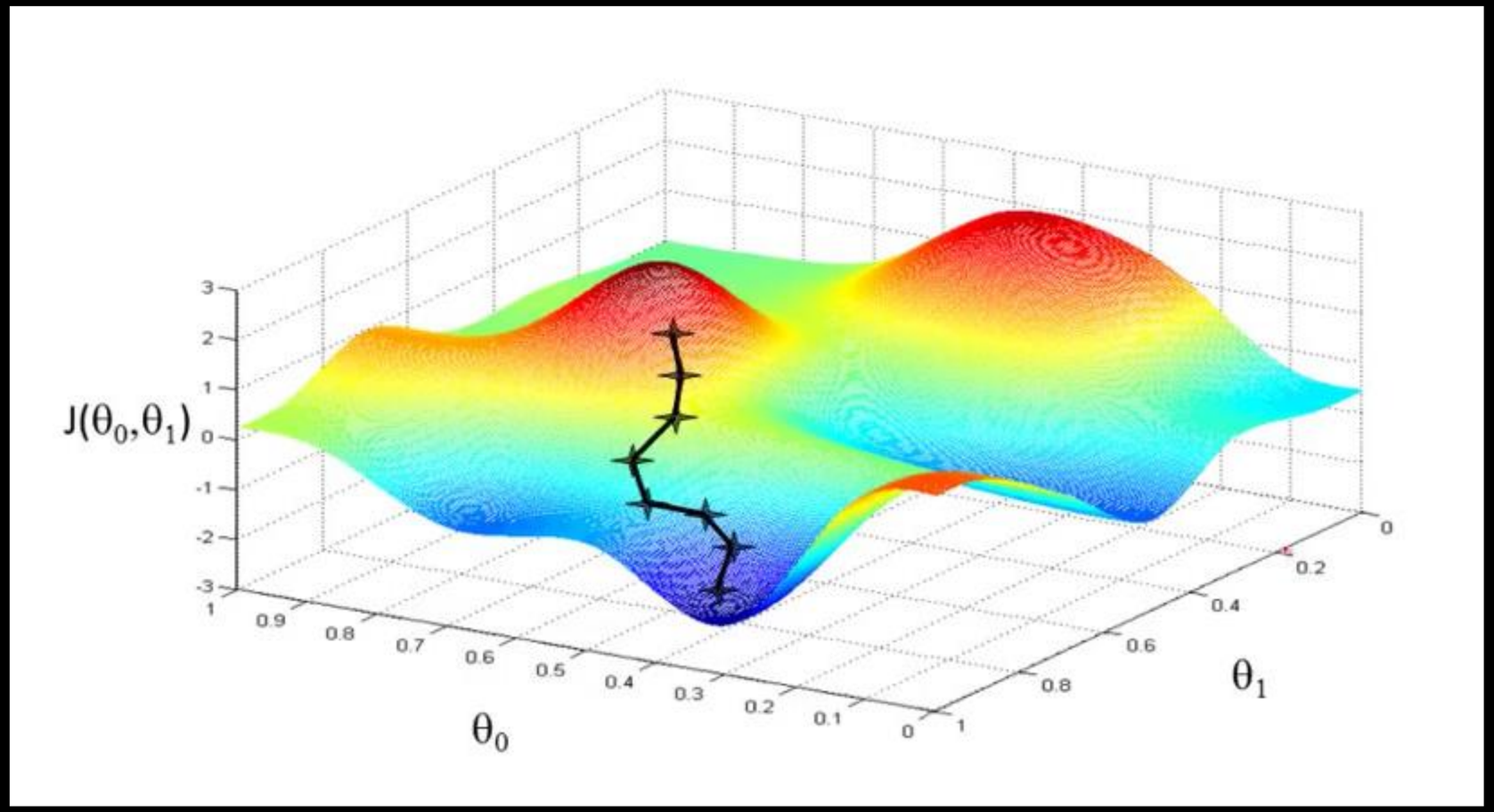
How much should this step outcome affect our weights and biases?

Momentum:

How much should past outcomes affect our weights and biases?

change = (learningRate * delta * value) + (momentum * pastChange)

PARAMETER VON OPTIMIZERN



- **Schrittgröße für die Annäherung an das Kosten-Minimum („Learning Rate“)**
- **Trägheit bei Richtungsänderungen („Momentum“)**

Quelle: https://www.coursera.org/learn/machine-learning

PARAMETER DES OPTIMIZERS „ADAM“

- Initialer Lernparameter (Schrittweite) für die Optimierung :
  alpha (learning rate)

- Anteil des aktuellen Gradienten in der Berechnung des nächsten Optimierungschritts:
  beta1 and beta2 (decay rates)

HYPERPARAMATER IN NEURONALEN NETZEN

- Wahl der Architektur:
  - Anzahl der Hidden Layer des Netzes
  - Typen der Hidden Layer
  - Anzahl der Neuronen je Hidden Layer
  - Wahl der Aktivierungsfunktionen
- Wahl der Kostenfunktion („Loss Function“)
- Wahl der Optimierungsfunktion („Optimizer“)
- Wahl der Learning Rate des Optimizers

DATENAUFBEREITUNG

Bei jeder Modellierung müssen die Daten folgende Eigenschaften haben:

1. **Es darf keine fehlenden Werte geben.**
2. **Alle Werte müssen Zahlen sein.**
3. **Kategoriale Variablen sind dummy-kodiert (one-hot-encoded).**

ONE-HOT ENCODING

id	color
1	red
2	blue
3	green
4	blue



id	color_red	color_blue	color_green
1	1	0	0
2	0	1	0
3	0	0	1
4	0	1	0

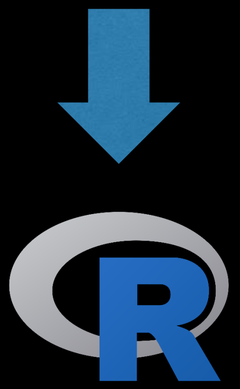
DATENAUFBEREITUNG FÜR DAS TRAINING DES NEURONALEN NETZES

```
1
2  ##################################################
3  ### Preparation of the Environment
22 ##################################################
23 ### Data Import
32 ##################################################
33 ### Data Preparation ####
34
35 # Preparation of independent variables ('features') by dummy coding the categorical variables
36 features <- as_tibble(model.matrix(price ~ as.factor(bathrooms) + as.factor(zipcode) + as.factor(condition) +
37 names(features)
38
39 # Construction of prepared data set
40 prepared_data <- tibble(label=data$price, features) %>%  # inclusion of the dependent variable ('label')
41     filter(complete.cases(.)) # Handling of missing values (here: only keeping rows without missing values)
42
43
44
45
46 ##################################################
47 ### Selection of Training, Validation and Test Data ####
48
49 # Set a random seed for reproducibility
50 set.seed(42)
51 # Shuffle the data
52 prepared_data_shuffled <- prepared_data %>% sample_frac(1)
53
54
55 # Calculate the number of rows for each dataset
56 n_total <- nrow(prepared_data_shuffled)
57 n_training <- floor(0.7 * n_total)
58 n_validation <- floor(0.20 * n_total)
59
60
61 # Split the features and labels for training, validation, and test
62 training_features <-
63   prepared_data_shuffled %>% select(-label) %>% slice(1:n_training)
64 validation_features <-
65   prepared_data_shuffled %>% select(-label) %>% slice((n_training + 1):(n_training + n_validation))
66 test_features <-
67   prepared_data_shuffled %>% select(-label) %>% slice((n_training + n_validation + 1):n_total)
```

URSPRÜNGE VON R VS. PYTHON

Statistische Verfahren außerhalb des ML

Mathematik/ Statistik und Disziplinen mit Anwendungsbereichen von Statistik (Ökonometrie, Psychometrie, Biometrie, ...)



Statistische Verfahren des ML

Angewandte Informatik und freie Wirtschaft mit Anwendungsbereichen von ML

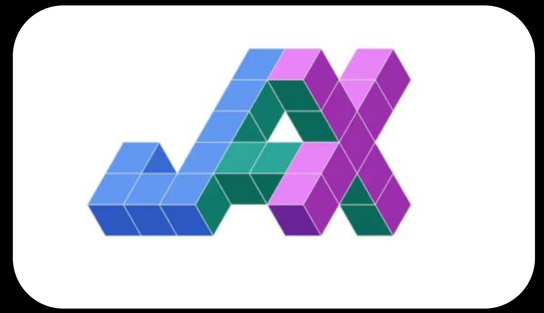


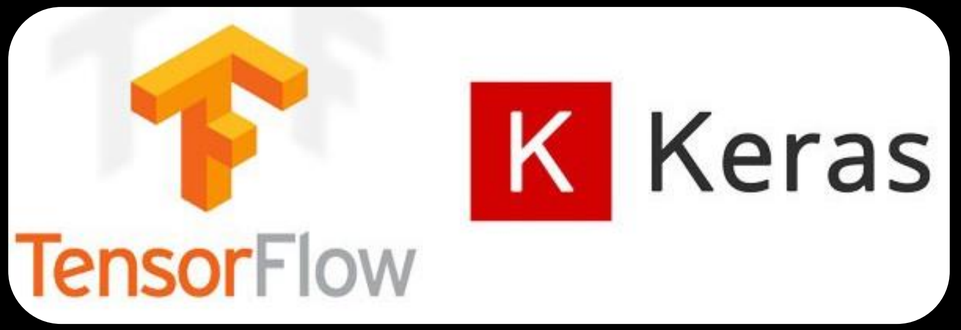

K
Keras


TensorFlow


scikit
learn


PYTORCH





Feb 2017: TensorFlow 1.0 (Estimator API)

Nov 2017: TensorFlow 1.4 (Estimator API, Keras API)

Jan 2019: TensorFlow 2.0 (Estimator API, Keras API)

Dez 2023: Keras 3.0
(unterstützt TensorFlow, Jax und PyTorch)

NUTZUNG VON KERAS IN RSTUDIO

Keras ist eine Schnittstelle (API/ ein Funktionswrapper) zur vereinfachenden Nutzung von TensorFlow.

Prinzipiell zwei Varianten :

- **Nutzung des Packages „keras“ (vgl. https://keras.rstudio.com/)**
- **Nutzung von Keras in Python und die Integration von Python über das Paket „reticulate“**

INSTALLATION VON PYTHON

```
---
title: "R Notebook"
output: html_notebook
---

### Installation von Python und der für TensorFlow benötigten Pakete (nur einmalig nö

```{r}
install.packages("reticulate")
library(reticulate)

# Installation von miniconda (falls nicht vorhanden)
install_miniconda(update=TRUE)

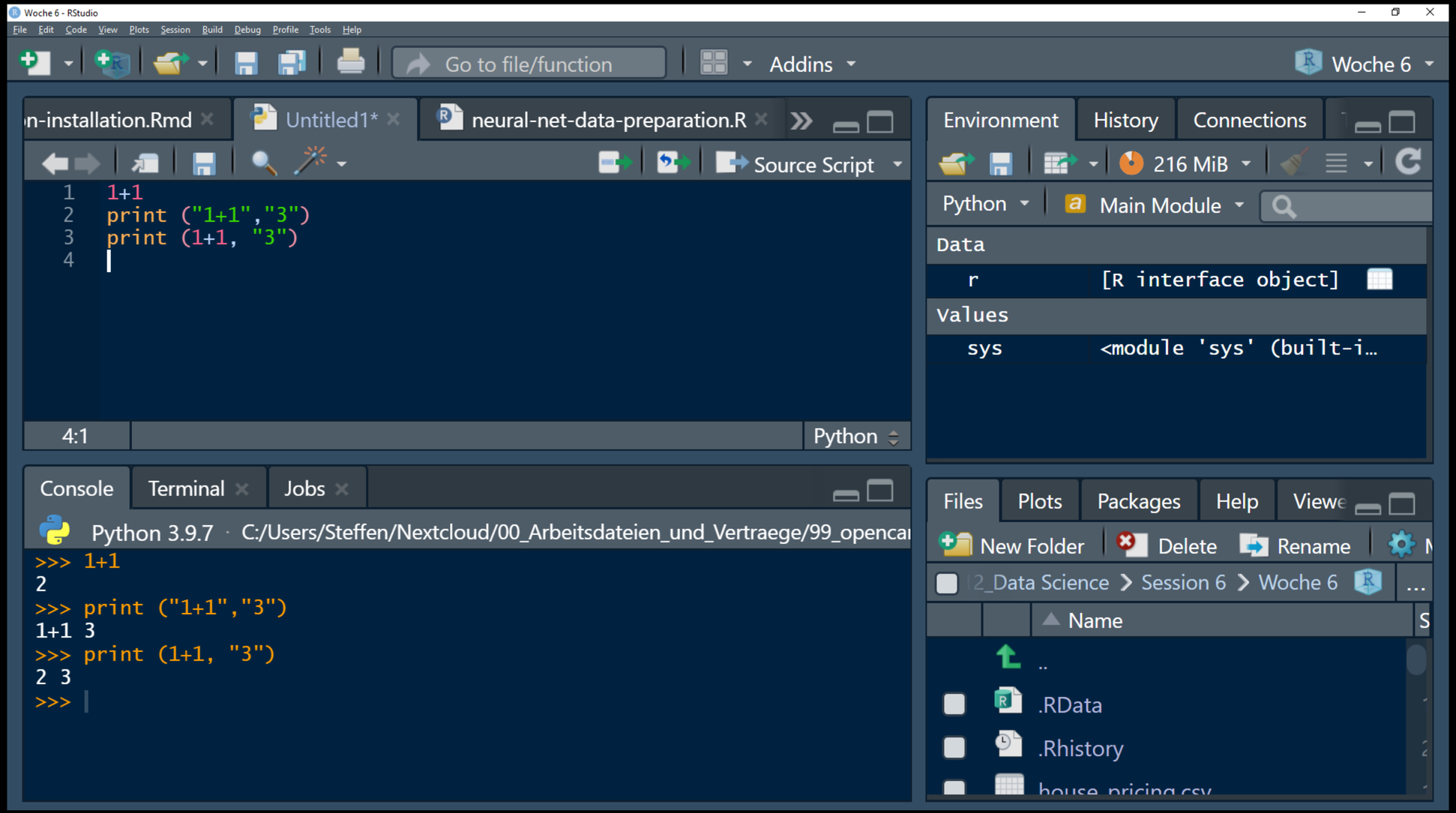

# Anlegen einer speziellen Python Umgebung
conda_create("r-reticulate", python_version = "3.8" )

# Installieren der Pakete in der angelegten Umgebung
conda_install("r-reticulate", "pandas")
conda_install("r-reticulate", "numpy")
conda_install("r-reticulate", "tensorflow")
conda_install("r-reticulate", "h5py")

# Verwenden der speziellen Python Umgebung die zuvor erstellt wurde
use_condaenv("r-reticulate")

```
```

VERWENDUNG VON PYTHON IN RSTUDIO



VERWENDUNG VON PYTHON IN KOMBINATION MIT R

````
```{python}
import sys
import tensorflow

# Augabe der installierten Python- und TensorFlow-Versionen
print("Python Version:      " + sys.version+"\nTensorFlow Version: "+tensorflow.__version__)

```

```{r}
# Import Libraries
library(reticulate)


# Importing Data
data <- mtcars

```


```{python}
mpg = r.data['mpg']

```


```{r}
table(py$mpg)

```
````

Siehe auch: https://rstudio.github.io/reticulate/articles/r_markdown.html

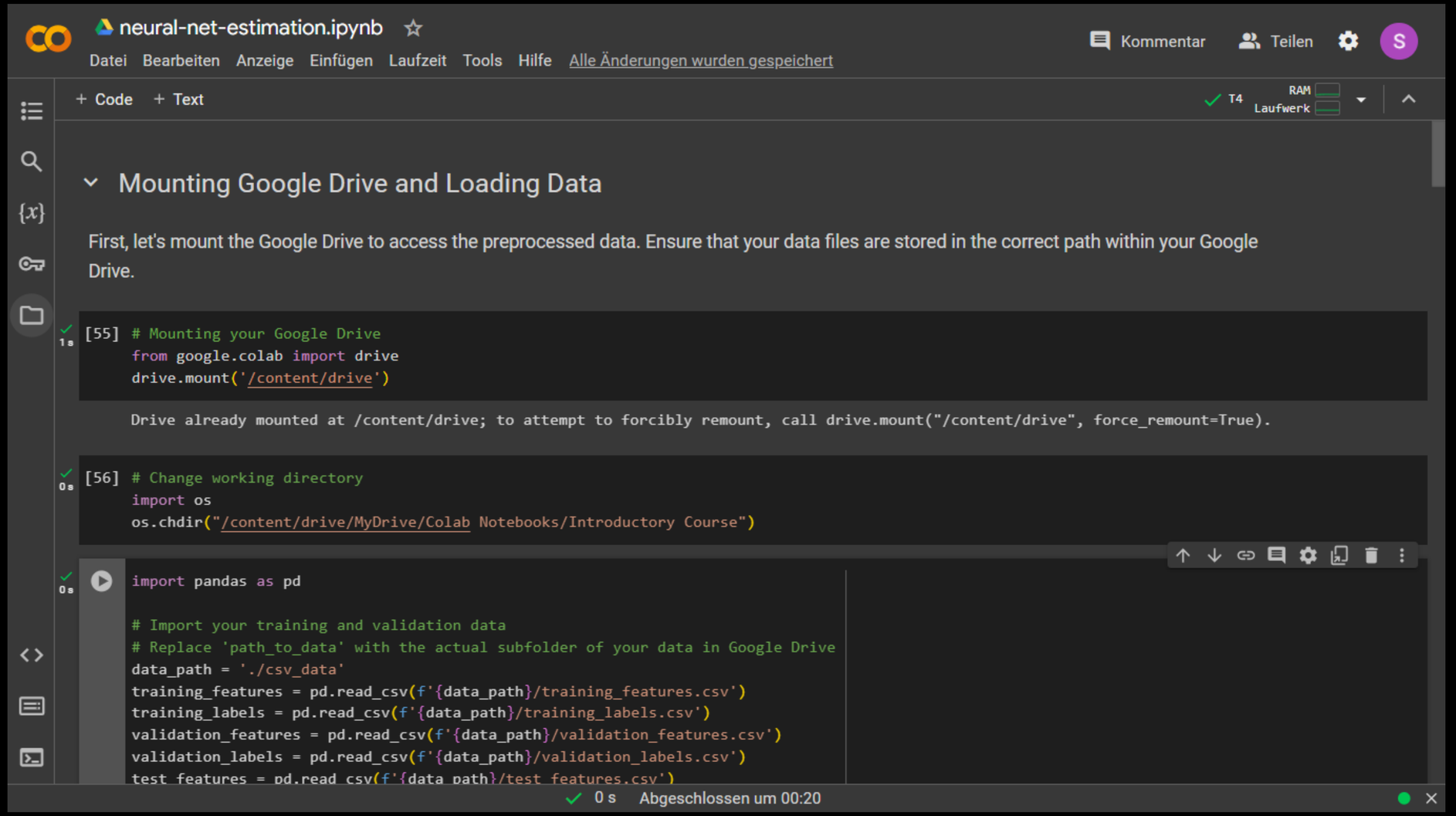
GOOGLE COLAB

- Cloud-basierte Plattform
- Kostenloser Zugang zu GPUs
- Vorinstallierte Bibliotheken
- Kollaborativ und teilbar
- Integration mit Google Drive

GOOGLE DRIVE

- Jeglicher Speicher in Colab ist temporär
- Direkter Zugriff aus Colab ermöglicht permanente Datenverwaltung
- Automatische Synchronisation und Backups

DEFINITION UND OPTIMIERUNG EINES NEURONALEN NETZES IN COLAB



BREAKOUT

- Ergänzt Eure Datenaufbereitung um die im Beispielskript durchgeführten Schritte:
  1) One-Hot-Kodierung kategorieller Variablen
  2) Entfernen von Fällen mit Missing Values
  3) Export der Trainings-, Validierungs- und Testdaten als CSV

- Öffnet das Jupyter-Notebook zur Schätzung des Neuronalen Netzes in Colab und speichert eine Kopie in Eurem Google Drive.

- Passt das Notebook an, um das neuronale Netzt für Eure Daten zu optimieren.

BATCH UND EPOCHE

Batch

- **Eine Menge von Fällen, die genutzt wird, um die Gewichte des Modells zu optimieren.**
- **Ein Optimierungsschritt wird auf Basis eines Batches durchgeführt.**

Epoche

- **Das Modell wurde einmal anhand aller Fälle bzw. aller zu einem Trainingsdatensatz existierenden Batches trainiert.**

Je nach Modell kann es notwendig sein, es über mehrere hundert oder tausend Epochen zu optimieren.

NORMALISIERUNG

Definition:

- **Abziehen des Mittelwerts und Teilen durch die Standardabweichung.**

Erleichtert es dem neuronalen Netz, die benötigten Parameter zu erlernen, indem alle Werte in einem ähnlichen Wertebereich liegen.

BATCH-NORMALISIERUNG

- Durchführung der Normalisierung auf Batch-Ebene

Zusätzliche Optimierung:

- exakt gleiche Mittelwerte und Standardabweichungen sind nicht notwendigerweise optimal im Sinne der Modellierung

→ Einfügen der Normalisierungsparameter als trainierbare Parameter

AUFGABEN

- Untersucht alle Eure Modellvariablen auf die Existenz von fehlenden und unplausiblen Werten
- Trainiert ein erstes neuronales Netz für Euren Datensatz (Löscht dazu zunächst alle Zeilen mit fehlenden Werten)
- Dieses Video (2 Minuten) zu Dropout-Layern schauen.
- Dieses Video (7 Minuten) als zusätzliche Einführung in die Definition von neuronalen Netzen schauen.